



MODULE 4

Swift Programming Fundamentals

Enterprise Mobile Application Development Bootcamp | AI-Integrated Edition

Module 4 at a Glance



1

**The Swift
Mental Model**



2

**Variables,
Types & Flow**



3

**Functions
& Closures**



4

**OOP in
Swift**



5

**Swift Safety
Features**



Lab

**Banking
Project**



Learning Objectives

- Explain why Swift is statically typed and the advantages over Python and JavaScript
- Declare variables and constants and choose correctly between let and var
- Write control flow: if/else, exhaustive switch, guard, and all loop types
- Define functions using labeled parameters, default values, and tuple returns
- Write closures and use map, filter, and reduce on collections
- Model domain concepts using classes, structs, enumerations, and protocols
- Explain Automatic Reference Counting and identify and fix retain cycles
- Safely unwrap optionals using if let, guard let, and optional chaining
- Write throwing functions and handle typed errors with do-catch
- Write generic functions and types constrained by protocol requirements



The Swift Mental Model

- Why Swift exists
- Static vs. Dynamic typing
- Safety as a design principle



Why Swift Exists



Type Safety

The compiler knows the type of every value at build time.
Type mismatches are build errors - not crashes in production.



Optionality is Explicit

A type either can or cannot be nil - and the contract is part of the type system, not a comment or convention.



Immutability by Default

let creates a true constant. The compiler prevents reassignment - no runtime check needed.



Errors Are Typed

Throwing functions declare that they can fail. Callers must handle every failure mode - the compiler enforces this.



Python/JS vs. Swift - The Core Differences

Python / JavaScript

x = 5; x = 'hello' # works

```
x = 5
x = 'hello' # no error
```

**None can appear anywhere
(convention, not enforcement)**

```
user = None # always valid
```

Swift

Static typing: type is fixed at declaration

```
var x = 5
x = "hello" // compile error
```

Optional<T> - nil must be declared

```
var user: String? // opt-in
var name: String  // never nil
```



Python/JS vs. Swift - Errors and Mutability

Python / JavaScript

Variable can be anything

```
x = 5  
x = 10  # fine, no warning
```

raise ValueError() - optional to catch

```
def login():  raise ValueError()  
login()  # no catch required
```

Swift

let = truly immutable constant

```
let x = 5  
x = 10  // compile error
```

throws - caller MUST handle it

```
func login() throws -> Token  
try login()  // must be in do-catch
```



Review

Skill Check ✓

- In one sentence: what does 'statically typed' mean?
- If a Swift variable is a non-optional String - what values can it hold?
- Name one type of bug that Swift would catch at compile time that Python would only catch at runtime.
- Why might a compiler error be BETTER than a runtime error?



Variables, Types & Control Flow

- let vs. var
- Type inference
- Strings
- Type conversion
- if/else
- switch
- guard
- Loops



let vs. var - Mutability is a Choice

- let - constant, cannot be reassigned
- Reach for let FIRST
- Switch to var only when mutation is required
- Compiler enforces this - cannot be forgotten
- var - variable, can be reassigned
- Use for values that must change
- Mutability communicates intent to readers

Code Bite

```
let appName = "PNC Mobile" // constant
var loginAttempts = 0      // variable

loginAttempts += 1         // allowed
appName = "Other"          // COMPILE ERROR

// Type inference - no annotation needed
let balance = 4_250.75     // inferred: Double
// Explicit annotation
var interestRate: Double = 0.035
// _ as thousands separator (readability)
let limit = 5_000_000
```



Type Safety and Explicit Conversion

```
// No implicit numeric conversion
let count: Int = 47
let total: Double = 12_309.88
// COMPILE ERROR: cannot convert Int to Double

let avg = total / count
// FIX: explicit conversion
let avg = total / Double(count)    // 261.91


// Optional conversion - safe, never crashes
let parsed = Int("2500")    // Int? - may be nil
let bad     = Int("abc")    // nil - no crash
if let amount = parsed {
    print("Amount: \(amount)")
} else {
    print("Invalid input")
}
```

Python does this

```
int('abc')
# → ValueError at RUNTIME

user says 'abc'
app crashes
```

Swift does this

```
Int("abc")
// → nil    (never crashes)

user says 'abc'
app handles gracefully
```



The Swift switch - Exhaustive Pattern Matching

- Exhaustive - must handle every case
 - Compiler error if you miss one
- Range patterns are possible
 - case 100...999: handles any Int in range
- Where clauses
 - case let x where x > 0 && x < 18:
- Tuple matching
- No fall-through by default
 - Use fallthrough keyword explicitly if needed

Code Bite

```
// Range matching - Python has no equivalent
switch creditScore {
case 800...850:  print("Exceptional")
case 740...799:  print("Very Good")
case 670...739:  print("Good")          // 714
lands here
case 580...669:  print("Fair")
default:         print("Poor")

// Remove default → COMPILE ERROR: not
exhaustive

}
```



switch Enums - Must Be Exhaustive

- Exhaustive - must handle every case
 - Compiler error if you miss one
- case (.checking, true): ...
- No fall-through by default
 - Use fallthrough keyword explicitly if needed

Code Bite

```
// Enum switching - no default possible if exhaustive
switch transactionType {
case .credit:    processCredit()
case .debit:     processDebit()
case .transfer:  processTransfer()
case .fee:       processFee()
// No default needed - enum IS exhaustive
}
```



guard - Early Exit and Clean Preconditions

Without guard (pyramid of doom)

```
func process(amount: Double?,
             balance: Double) {
  if let amt = amount {
    if amt > 0 {
      if amt <= balance {
        // work buried 3 levels deep
        executeTransfer(amt)
      } else {
        print("Insufficient")
      }
    } else {
      print("Invalid amount")
    }
  } else {
    print("No amount")
  }
}
```

With guard (flat, readable)

```
func process(amount: Double?,
             balance: Double) {
  guard let amt = amount else {
    print("No amount")
    return // exit here
  }
  guard amt > 0 else {
    print("Invalid amount")
    return // exit here
  }
  guard amt <= balance else {
    print("Insufficient")
    return // exit here
  }

  // work at LEFT MARGIN, always reached
  executeTransfer(amt)
}
```



Variables, Types, and Control Flow

Skill Check ✓

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/VariablesTypesFlow directory.



Review

Skill Check ✓

- What error do you get if you add an Int and a Double without conversion? How do you fix it?
- You write a switch on an Int. You have cases for 1, 2, and 3. What must you add? What happens if you don't?
- Rewrite this in your head using guard: if let x = optional { if x > 0 { doWork() } }
- What is the difference between for-in with a where clause and calling .filter before iterating?



Functions & Closures

- Labeled parameters
- Default values
- Tuple returns
- Closures
- map/filter/reduce
- Capture lists



Labeled Parameters - Self-Documenting Call Sites

- By default, when calling a function, the parameter names must be used when passing values
- This makes the invocation more readable

Code Bite

```
func transfer(amount: Double,  
    source: String,  
    destination: String) -> Bool {  
    print("\(source) → \(destination):  
$\(amount)")  
    return true  
}  
  
// Call site reads like prose  
transfer(amount: 500.00, source: "Checking",  
    destination: "Savings")
```



Internal vs. External Names

- You can set up the function so that you can use a different parameter name when invoking the function
- When defining the function, list the external name first and then the internal name
 - Separate them with a space

Code Bite

```
func transfer(amount: Double,  
  from source: String,  
  to destination: String) -> Bool {  
  // inside: use internal names  
  print("\(source) → \(destination):  
$\(amount)")  
  return true  
}  
  
// Call site uses external names  
transfer(amount: 500.00, from: "Checking",  
  to: "Savings")
```



Labeled Parameters - Suppressing Label Name

- Occasionally, you may want to invoke a function without providing the parameter name
- In this case, use an underscore ("_") as the external name

Code Bite

```
// Suppress first label with _  
func authenticate(_ username: String,  
    with password: String) -> Bool {  
    return username == "jsmith"  
        && password == "pass123"  
}  
  
authenticate("jsmith", with: "pass")  
// clean call site
```



Parameter Default Values

- When defining a function, you can give any of its parameters a default value
- This makes the parameter optional

Code Bite

```
// Default values
func fetchTransactions(limit: Int = 50,
    offset: Int = 0) -> String { ... }

fetchTransactions()
// limit=50, offset=0

fetchTransactions(limit: 20)
// limit=20, offset=0

fetchTransactions(limit: 20, offset: 40)
// explicit both
```



Named Tuple Returns

- If you want to return more than one piece of data from a function, you can create a named tuple as the return type
- For the return type surround the multiple names/data types in parenthesis

Code Bite

```
func validate(amount: Double) ->
    (isValid: Bool, error: String?) {
    guard amount > 0 else {
        return (false, "Must be > 0")
    }
    return (true, nil)
}

let r = validate(amount: 500)
if r.isValid {
    proceed()
} else {
    show(r.error!)
}
```



Closure Syntax - From Full to Shorthand

1

Full form

```
numbers.sorted(by: { (a: Int, b: Int) -> Bool in  
    return a < b  
})
```

2

Implicit return

```
numbers.sorted(by: { (a: Int, b: Int) -> Bool in a < b })
```

3

Inferred types

```
numbers.sorted(by: { a, b in a < b })
```

4

Trailing closure

```
numbers.sorted { a, b in a < b }
```

5

Shorthand arguments

```
numbers.sorted { $0 < $1 }
```



Functional Collection Processing - iterating

- .forEach() will execute some behavior for each element in the array
- Void side effects only, there is no return

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,  
8_900.00, 125.50, 22_450.00]
```

```
balances.forEach( { (bal: Double) in  
    print("The balance is $\(bal)")  
} )
```

```
// or
```

```
balances.forEach {  
    print("The balance is $\( $0 )")  
}
```



Functional Collection Processing - filter

- .filter() will build a new array consisting of a subset of the original elements
- The closure should return a Boolean
 - Elements whose execution of the closure returns true will be included
- Returned is an array of the same type, with potentially fewer elements

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,  
8_900.00, 125.50, 22_450.00]  
  
// filter - returns [T] of elements where  
// closure returns true  
let highValue = balances.filter { $0 > 5_000  
}  
  
// [12000.0, 8900.0, 22450.0]
```



Functional Collection Processing - map

- You pass in a closure that translates one of the existing items from the array into the value that you wish included in the resultant array
- .map() will build a new array
 - It will contain the return values from all the invocations of your closure

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,  
8_900.00, 125.50, 22_450.00]
```

```
// map - transforms each element
```

```
let withInterest = balances.map {  
  $0 * 1.035  
}
```

```
// each balance increased by 3.5%
```



Functional Collection Processing - reduce

- `.reduce()` combines all the array values into one value
- Its parameters include the initial value for the summary and the closure you wish to use
- The closure accepts two parameters
 - The summary so far
 - The current value being combined
- The closure returns the modified summary
 - This value will be used on the next iteration
 - The last iteration's return will be the final return value from `reduce()`

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,  
8_900.00, 125.50, 22_450.00]
```

```
// reduce - collapses to a single value
```

```
let total = balances.reduce(0, +)
```

```
// 47176.25
```

```
let total2 = balances.reduce(0) { $0 + $1 }  
// same thing
```



Chaining Calls

- Most of the array methods return a new array
- Calls can be chained together in a Fluent style

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,
8_900.00, 125.50, 22_450.00]

// Chain - filter then map then reduce
let premiumTotal = balances
    .filter { $0 > 1_000 }
    // keep only large balances
    .map { $0 * 1.035 }
    // apply 3.5% interest
    .reduce(0, +)
    // sum everything
```



Functional Collection Processing - sorting

- `.sorted()` returns a new array with the same elements in a different order
- The closure you provide should return a Boolean
 - It will tell the algorithm whether the items being compared are in the right order or not

Code Bite

```
let balances = [3_250.00, 12_000.00, 450.75,  
8_900.00, 125.50, 22_450.00]
```



Functions and Closures

Skill Check ✓

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/FunctionsClosures directory.



Review

Skill Check ✓

- What is the external label on a function parameter and what is the internal label? Give an example of each.
- What does `$0` represent in a closure? What does `$1` represent?
- You call `.map { $0 * 2 }` on `[1, 2, 3]`. What does it return? What type is it?
- Why would you use `.reduce` instead of a for loop with an accumulator variable?
- When should you add `[weak self]` to a closure capture list?



Object-Oriented Swift

- Structs (value types)
- Classes (reference types)
- Enumerations
- Protocol-Oriented Programming



struct Property Syntax

- struct is a value type
- Memberwise init is FREE - no need to write init()

Code Bite

```
struct Transaction {  
    let id: String  
    let date: Date  
    let amount: Double  
    var description: String  
    let isDebit: Bool  
}  
  
let tx = Transaction(id: "001",  
    date: Date(), amount: 500,  
    description: "Deposit", isDebit: false)
```



struct Computed Properties

- Computed property - derived from stored data
- Define a function body after the variable declaration
- Property values can be accessed by directly working with the variables

Code Bite

```
struct Transaction {  
    ...  
    var formattedAmount: String {  
        let prefix = isDebit ? "-" : "+"  
        return "\(prefix)$\n(String(  
            format: "%.2f", amount))"  
        }  
    }  
}  
  
let tx = Transaction(id: "001",  
    date: Date(), amount: 500,  
    description: "Deposit", isDebit: false)  
print(tx.formattedAmount)    // +$500.00
```



struct Mutator Methods

- Any method that changes a struct property must have the mutating modifier on it
- The property that gets changed must be declared as a var and not a let constant

Code Bite

```
struct Transaction {  
    ...  
    mutating func addNote(_ note: String) {  
        description = description + " (\(note))"  
    }  
}
```



struct is a Value Type

- Assigning one struct to a second variable results in independent copies
- This is not as inefficient as you may imagine
- Swift operates by a copy-on-write strategy, so a copy is not created until a property is changed

Code Bite

```
var t1 = Transaction(...)
var t2 = t1           // copy

t2.description = "Modified"

// t1 is UNCHANGED
print(t1.description)
// "Deposit" ← original

print(t2.description)
// "Modified" ← copy
```



class - Property Syntax

- Classes are reference types
- Multiple variables assigned the same object will be references to one instance

Code Bite

```
class BankAccount {  
    let id: String  
    let accountNumber: String  
    var balance: Double  
    let owner: String  
  
    init(id: String, acctNum: String,  
        bal: Double, owner: String) {  
        self.id = id  
        self.accountNumber = acctNum  
        . . .  
    }  
}
```



class - Initialization and Deallocation

- Classes must have an initializer
 - Or else initialize all properties to some default value
- Automatic Reference Count (ARC) signal is the deinit() method
- Properties must be accessed through the self keyword

Code Bite

```
class BankAccount {  
    ...  
    init(id: String, acctNumber: String,  
owner: String, balance: Double = 0.0) {  
        self.id = id  
        self.accountNumber = acctNumber  
        self.balance = balance  
        self.owner = owner  
    }  
    deinit {  
        print("\(accountNumber) deallocated")  
    }  
}
```



class - Behavioral Methods

- Methods can access and/or modify property values
- Must use the self keyword to access them

Code Bite

```
class BankAccount {  
    ...  
    func deposit(amount: Double) {  
        guard amount > 0 else { return }  
        self.balance += amount  
    }  
    func withdraw(amount: Double) -> Bool {  
        guard amount > 0, amount <= balance else  
        { return false }  
        self.balance -= amount  
        return true  
    }  
}
```



class - Reference Semantics

- Assigning the value of one object reference to another variable creates a second reference to the same object instance
- Changes made to properties through one reference will be seen by the other reference(s)

Code Bite

```
let acc = BankAccount(balance: 1000)
let ref = acc    // SAME object

ref.deposit(amount: 500)

// BOTH show 1500 - same object
print(acc.balance)    // 1500.0
print(ref.balance)    // 1500.0
```



struct vs. class - Rule of Thumb

- Use struct for DATA that represents a VALUE:
 - Transaction, Address, UserProfile, Color.
- Use class for OBJECTS with IDENTITY and LIFECYCLE.



Enumerations - Typed, Exhaustive, Powerful

- Can have raw values of any type
 - e.g. to map to external API strings
- Exhaustive switch behavior
- Compiler error if any case is unhandled
- Adding a new case forces updates everywhere

Code Bite

```
enum AccountType: String {  
    case checking    = "CHECKING"  
    case savings     = "SAVINGS"  
    case investment  = "INVESTMENT"  
    case credit      = "CREDIT"  
  
    var displayName: String {  
        switch self {  
            case .checking: return "Checking Account"  
            case .savings: return "Savings Account"  
            case .investment: return "Investment Account"  
            case .credit: return "Credit Card"  
        }  
    }  
}
```



Enumeration Cases Can Have Different Typed Data

- Associated values - carry different data per case
- Associated values
- Each case carries different typed data
- Replaces dict/tuple anti-patterns

Code Bite

```
enum AccountError {  
  case insufficientFunds(  
    available: Double, requested: Double)  
  case accountInactive  
  case dailyLimitExceeded(limit: Double)  
}
```



Enumerations - Typed, Exhaustive, Powerful

- Switch statements need to handle all possible values
- It is considered a best practice to NOT use default
 - This will force developers to visit each use of switch when a new enum value is created

Code Bite

```
switch error {  
  case .insufficientFunds(let avail,  
                                let req):  
    print("Need $\(req), have $\(avail)")  
  
  case .dailyLimitExceeded(let limit):  
    print("Limit is $\(limit)")  
  
  case .accountInactive:  
    print("Account inactive")  
}
```



Protocols Are Contracts

- Protocol defines a contract
- Structs AND classes can conform
- A type can adopt multiple protocols
- No coupling to a base class
- They are preferred in Swift

Code Bite

```
protocol Summarizable {  
    var summary: String { get } // required  
    func printSummary()         // required  
}
```



Extensions can Provide Implementation

- Extensions can add default behavior to an existing protocol
 - Conforming types will automatically acquire the behavior
- No conforming type needs to implement
 - They can, however, override the default behavior

Code Bite

```
// No conforming type needs to implement  
extension Summarizable {  
    func printSummary() { print(summary) }  
}
```



Structs can Adopt Protocols

- Structs can conform to any protocol
- They must implement the properties defined in the protocol

Code Bite

```
struct Transaction: Summarizable {  
    // ... properties ...  
    var summary: String {  
        "\((formattedDate) \((description)  
        \((formattedAmount))"  
    }  
  
    // printSummary() inherited from  
    // the extension  
}
```



Classes can Adopt Protocols

- Classes can conform to any protocol
- They must implement the properties defined by the protocol

Code Bite

```
class BankAccount: Summarizable {  
    // ... properties ...  
  
    var summary: String { "\(displayName)  
    \formattedBalance)" }  
  
    // printSummary() inherited from  
    // the extension  
  
}
```



Protocols are Data Types

- The protocol data type can be used as a function parameter
- It can also be used as the return type from a function

Code Bite

```
func printAll(_ items: [Summarizable]) {  
    items.forEach { $0.printSummary() }  
}
```

```
let transaction1 = Transaction(. . .)  
let transaction2 = Transaction(. . .)  
let account = BankAccount(. . .)
```

```
printAll([transaction1,  
         transaction2, account])
```



Protocols vs. Inheritance

- Protocol
 - Structs AND classes can conform
 - A type can adopt multiple protocols
 - No coupling to a base class
 - Extensions add default behavior
 - They are preferred over Inheritance
- Inheritance
 - Classes only
 - Single parent class
 - Tight coupling to base behavior
 - Use when you need override capacity



Object Oriented Swift

Skill Check ✓

- Find and follow the Swift code file in the GitHub repository for the Phase2/Module04/ObjectOrientedSwift directory.

